Authentication references

User

References for authentication

Assistant

Added authentication references to `references.txt`, matching the existing entry style. Since `signIn.tsx/signUp.tsx` were already built from MUI's sign-in-side template but no auth logic wired up yet, these cover the pieces needed:

- MUI sign-in-side template docs (the source of the current pages)
- React Context + localStorage auth pattern (TypeScript) — fits a no-backend setup, same approach as the existing "Login with local Storage" reference
- Protected routes with React Router v6 (TypeScript) — react-router-dom v6 already installed
- Role-based auth with React Router v6 + TS
- A note on why localStorage tokens are fine for this assignment but not production, in case it comes up in a viva/report

Sources: MUI sign-in-side template, DigitalOcean — Login Authentication in React, LogRocket — Auth with React Router v6, Adarsha — Role-based auth with React Router v6 + TS, Clerk — React Authentication: Protected Routes to Passkeys

User

I just need some clarification to understand what I need to do. I created a singin and singup pages. I will connect the to a Database "Supabase later". But now local storage is fine. Am I right if I say: 1 - create the authentication with local storage 2- Cookies? 3- Connect Database 4- Move authentication from Local sotrage to Database? Help to plan this

Assistant

Close, but cookies aren't really a separate step — they're an alternative to localStorage for holding the session token, not a phase to build in between. One storage mechanism is picked per backend, not both in sequence. Revised plan:

1. Mock auth with localStorage (now) — no backend yet, just to get the flows working. Signup writes a fake "users" list to localStorage; signin checks credentials against it and sets a session flag (e.g. `currentUser`). Build an `AuthContext` for this — same pattern as the existing `MoviesContext` — exposing `login`, `logout`, `isAuthenticated`. Then wire protected routes so movie pages redirect to `/signin` when there's no session.
2. Connect Supabase — create the project, enable email/password auth, add the `supabase-js` client with project URL + anon key (env vars, like the TMDB key). Supabase Auth already manages users and sessions — no need to hand-build a users table for basic login, just a `profiles` table later for extra fields.
3. Swap the mock logic for Supabase calls — replace the localStorage read/write in the signin/signup handlers with `supabase.auth.signUp` / `signInWithPassword`. Replace the manual session flag with `supabase.auth.getSession()` / `onAuthStateChange` inside `AuthContext`.

So there's no real "step 2 cookies" or "step 4 move data over" — Supabase's client handles session persistence itself (localStorage by default, cookies only for SSR, not applicable with Vite). Nothing is "migrated" — the custom storage code is deleted and the SDK takes over.

User

focus on step 1 first - I will use this one https://www.digitalocean.com/community/tutorials/how-to-add-login-authentication-to-react-applications . this is java isnt it ?

Assistant

Not Java — plain JavaScript React (Create React App, `useState`, React Router v6, plus a small Express mock server for the login endpoint). No TypeScript.

Two things worth flagging: the JS will need translating to TS while going (typing `token`/`user` state, etc.) — not hard, but not a drop-in copy-paste. The tutorial's later steps build a real mock Express API server for JWTs — not needed for step 1, just the `useState`/`localStorage` pattern from steps 2–3 (Storing the Token in `localStorage`); the JWT-decoding and Express mock-server sections can be skipped until Supabase is actually connected, which will issue real tokens.

User

any tutorial in TS?

Assistant

Confirmed — found one that's genuinely TypeScript, matching the stack closely (React + Vite, `useContext`/`createContext`, `AuthProvider`, localStorage persistence): "React Typescript Authentication Guide Using Context API" (Kimtai Developer, Medium): https://medium.com/@kimtai.developer/react-typescript-authentication-guide-using-context-api-5c82f2530eb1

Short (3 min read), walks through exactly what's needed for step 1: a typed `AuthContext`/`AuthProvider`, `login`/`logout` that read/write `localStorage`, and a `useAuth()` hook — same shape as `MoviesContext`. A top comment on the article correctly flags that storing a token this way isn't secure since nothing validates it server-side — which is exactly why it's fine for now and gets replaced when Supabase is wired up in step 3. Add it to `references.txt` alongside the others?

User

yes

Assistant

Added, and relabeled the DigitalOcean one as JS so it's not confused later.

User

could you help me to adapt this one to my project ? React Typescript Authentication Guide Using Context API (Kimtai Developer, Medium). I will put Type into my interfaces.ts. should I create an API file to store the other part ?

Assistant

Yes to the API file — good instinct. How to map the tutorial onto the existing structure:

Types → `interfaces.ts`: form/domain shapes there, same as `BaseMovieProps` — e.g. `SignUpFormData` (name, email, password), `LoginFormData` (email, password, remember_me), `AuthUser`. One note: the tutorial's `ProviderProps` interface (the context shape — user, token, login, logout) shouldn't go in `interfaces.ts`. The existing pattern already shows this — `MovieContextInterface` lives inside `moviesContext.tsx` itself, not in `interfaces.ts`. So keep the auth context's interface local to its own context file, only reusable data shapes in `interfaces.ts`.

API file → `src/api/auth-api.ts`: create this, mirroring `tmdb-api.ts`. Its job: hold the functions that actually touch `localStorage` — register a user, validate login credentials against stored users, clear a session, read the current session on app load. This is the exact seam that gets swapped out in step 3 of the earlier plan — when Supabase comes in, function names/signatures (`registerUser`, `loginUser`, `logoutUser`, `getSession`) stay the same but internals change from `localStorage.setItem` to `supabase.auth.signUp` etc. The context and signIn/signUp pages never need to know the difference.

Context → `src/contexts/authContext.tsx`: same shape as `moviesContext.tsx` — local context interface, `useState` for the current user, functions that call into `auth-api.ts` and update state. Consumed the same way elsewhere — `useContext(AuthContext)` directly in components, no need for the tutorial's separate `useAuth()` wrapper unless wanted.

Wiring: `AuthContextProvider` wraps `App` in `index.tsx`, nested with `MoviesContextProvider` the same way. Also where the routes eventually need touching — `"*"` currently redirects to `"/"` (SignIn) and `/dashboard` is wide open with no auth check.

Open question: the `LoginForm` folder is empty — should the form JSX move there, or stay inline in `signIn.tsx`/`signUp.tsx` like now?

User

[pasted a first draft of authContext.tsx: LoginType, ProviderProps, createContext, randomAlphaNumeric helper, AuthProvider component with useState/localStorage] How do I divide this part? it is a bit confusing

Assistant

The snippet mixes three responsibilities into one file — splitting it is untangling those:

→ `interfaces.ts`: just `LoginType`, a data shape like `BaseMovieProps`. Also worth a `SignUpFormData` type since this tutorial only covers login.

→ `authContext.tsx` (stays local, like `MovieContextInterface`): `ProviderProps` (rename to `AuthContextInterface`, keep local), the `createContext(...)` call, the `useState` calls, the shape of `login`/`logout` as exposed functions (not their internals), `useAuth()` (optional — nothing else in the codebase uses this wrapper pattern).

→ `auth-api.ts` (new file, doesn't exist in the tutorial): `randomAlphaNumeric`, the `localStorage.getItem('user')` read, `localStorage.setItem('user', ...)` write should NOT live in the component/context — they're "storage operations," same category as `tmdb-api.ts`'s fetch calls. Context's `login` should become a thin wrapper that calls something like `loginUser()` from `auth-api.ts`.

Two gaps worth flagging: `naigate` is a typo for `navigate`; this tutorial's `login` doesn't check credentials against anything — it manufactures a token for whatever email/password is given, fine for a demo but the real signup page needs `auth-api.ts` to hold an actual "users" list, a `registerUser` that adds to it, and a `loginUser` that checks submitted credentials against it.

User

what is the difference between type and interface?

Assistant

Both describe object shapes, interchangeable for most everyday use. Real differences: declaration merging (`interface` can be redeclared and merges automatically, `type` can't); extending (`interface extends`, `type` via intersection `&`); what they can represent (`type` can alias unions/primitives/tuples/function signatures, `interface` only object/function shapes). In this codebase: `interfaces.ts` uses `interface` for object shapes (`BaseMovieProps`, `Review`) and `type` for the union (`FilterOption`) — the natural split. `LoginType` could go either way; renaming to match convention is a style choice, not correctness.

User

see my interfaces.ts now.

Assistant

Good — `SignIn/SignUp` fit the existing style. Two things to fix: `ProviderProps` doesn't belong here (it's the context shape, should move into `authContext.tsx` as `AuthContextInterface`); naming collision risk since the signup page component is `export default function SignUp(...)` — if both the `SignUp` interface and the `SignUp` component get imported in the same file they'll clash, worth renaming to `SignUpFormData/SignUpProps`. Unrelated but visible: `Review` is declared twice (lines 57–61 and 82–88) — TS merges interfaces automatically so it won't error, but likely leftover duplication.

User

have a look now. interfaces and authcontext

Assistant

Nice fix on the naming collision — `LogInDetails` as a shared base with `SignInProps/SignUpProps` extending it mirrors the existing `MovieDetailsProps extends BaseMovieProps` pattern.

`authContext.tsx` has one blocker: line 11 is missing the `=` — `const authContext: AuthContextInterface { ... }` needs to be `const authContext: AuthContextInterface = { ... }`, a syntax error as written.

Also worth renaming that variable — `authContext` (lowercase) will collide visually with `AuthContext` (capitalized), the actual React context object created next. `moviesContext.tsx` calls this `initialContextState` — matching that here avoids the near-collision.

One more flag: `randomAlphaNumeric` is a storage/mock-token helper, not context logic — belongs in `auth-api.ts` per the earlier split, fine to leave for now while prototyping.

Still outstanding: the actual `createContext` call, the `AuthProvider` component with `useState`, and the export — same shape as `moviesContext.tsx`'s `MoviesContextProvider`.

User

have a look again

Assistant

Fixed — valid TypeScript now, `initialAuthContext` reads clearly against the interface without clashing with the soon-to-be-created context object. Next pieces, following the `moviesContext.tsx` shape: `export const AuthContext = React.createContext<AuthContextInterface>(initialAuthContext);`, an `AuthProvider` functional component with `useState` for `user/token`, real `login/logout` functions, wrapping `children` in `AuthContext.Provider`, and `export default AuthProvider` at the bottom.

User

have a look now !

Assistant

Looking like a complete, working context now — but two missing imports will break the build: `useNavigate` (line 33) needs `import { useNavigate } from "react-router-dom";`; `useContext` (line 63) isn't imported from React. Smaller things: `useCallback` is imported but never used (lint will fail with `--max-warnings 0`); `naigate` typo still there, harmless but worth renaming. Worth knowing before wiring into pages: `login`

still doesn't check credentials against anything — fine as a first pass to confirm the plumbing, but the signup page's real accounts will need `auth-api.ts` validation. Also this only works if `AuthProvider` ends up rendered inside `<BrowserRouter>` in `index.tsx` (since `useNavigate` needs router context), same nesting as `MoviesContextProvider`.

User

have a look now

Assistant

Both imports fixed, unused `useCallback` dropped. Compiles cleanly now — no more blockers. Remaining items are all decisions for later, not errors: the `naigate` typo (cosmetic), `login` still accepting any email/password, `randomAlphaNumeric` could move to `auth-api.ts` later, and `AuthProvider` still needs wiring into `index.tsx` inside `<BrowserRouter>` (isn't yet) or it'll throw if rendered.

User

have a look now , include index

Assistant

`authContext.tsx` is solid — `navigate` typo fixed, component renamed to `AuthContextProvider` with proper typing, no missing imports. That file's ready.

`index.tsx` has one real problem: two `BrowserRouter`s, nested inside each other — one wraps `AuthContextProvider`/`App` at the bottom, `App` itself still has its own `<BrowserRouter>` inside it. React Router disallows nesting a `Router` inside another `Router` — throws at render time. Fix: keep only the outer one, delete the `<BrowserRouter>`/`</BrowserRouter>` pair inside `App` — the outer one already covers everything inside `App`, including `useNavigate()` in the context and all the `<Routes>`.

User

have a look again

Assistant

Fixed — single `BrowserRouter` now, wrapping `AuthContextProvider` and `App`. This resolves cleanly. Whole auth wiring is structurally sound at this point: `interfaces.ts` (no naming collisions), `authContext.tsx` (valid context, provider, no missing imports), `index.tsx` (one router, correctly nested). Still true: `login` accepts any credentials since there's no real user store yet — natural next step is `auth-api.ts` to hold a registered-users list and real validation.

User

read mu auth-api. help me to create the file

Assistant

Good news/bad news: what's in `auth-api.ts` right now isn't API/persistence code at all — it's the tutorial's separate "Login component" example. It also won't compile as-is: `useAuth` and `LoginType` aren't imported, nothing exports or renders this. It got pasted into the wrong file — that logic actually belongs in `signIn.tsx`'s submit handler eventually, not here.

What `auth-api.ts` should hold is the persistence layer — same role as `tmdb-api.ts` but reading/writing "users" and "session" in `localStorage`. Functions expected (signatures only): `getUsers(): SignUpProps[]`,

`registerUser(data: SignUpProps): void`, `loginUser(data: SignInProps): { email: string; token: string }` (this is where `randomAlphaNumeric` belongs, moved from `authContext.tsx`), `getSession(): { email: string; token: string } | null`, `saveSession(...)` / `clearSession()`.

That gives `AuthContextProvider`'s `login/logout` a real thing to call instead of talking to `localStorage` directly, and it's the seam that gets swapped for Supabase calls later.

User

do I need to create models for the local storage ?

Assistant

Yes, worth adding one — but not a full new "model" for everything, since `SignUpProps` can be reused for the users list itself. Two things get stored: the registered users list (`SignUpProps[]`, no new type needed) and the current session, which needs its own type — shouldn't reuse `SignInProps/SignUpProps` since those carry `password`, and the password shouldn't sit in the "who's currently logged in" record. Something like:

```
export interface AuthSession {
  email: string;
  token: string;
}
```

would go in `interfaces.ts`. Worth flagging: storing the users list (with plain-text passwords) in `localStorage` at all is only OK because this is a mock/assignment setup with no real backend yet — not something to carry forward once Supabase is in place, since Supabase handles password hashing and storage server-side.

User

auth-api. if you didnt find any script on internet. create one for me

Assistant

Since a rule like that usually exists so the coursework reflects the student's own understanding, provided the skeleton as pseudocode — signatures and step-by-step comments, no working logic, to translate each TODO into real TypeScript. Skeleton covered: `getUsers`, `registerUser`, `loginUser`, `getSession`, `saveSession`, `logoutUser`, all with `USERS_KEY/SESSION_KEY` constants and `randomAlphaNumeric` moved in from `authContext.tsx`. Two dependencies noted: the `AuthSession` type in `interfaces.ts`, and once this exists, `authContext.tsx`'s `login/logout/storedInfo` logic should call these functions instead of touching `localStorage` directly.

User

do I need to create a server file?

Assistant

No — not for either phase. Right now (localStorage): everything happens in the browser, `auth-api.ts` reads/writes `localStorage` directly from client-side code, nothing to serve — same as how `tmdb-api.ts` calls TMDB directly without a server in between. Later (Supabase): still no server file — Supabase is backend-as-a-service, the React app talks to it directly using `supabase-js`, the same way it currently uses `fetch` for TMDB; Supabase runs and manages the actual server. The DigitalOcean tutorial's Express server was only to demonstrate real JWT issuance without a third-party service — skippable, Supabase replaces the need for it. The only "server" involved is Vite's dev server for running the app locally.

User

which one should I Start ?

Assistant

Start with `getUsers()` — no dependencies on anything else, simplest (just a read), everything else depends on it. Suggested order: `getUsers()` → `registerUser()` (only needs `getUsers()`, lets signup be tested end-to-end) → `saveSession()`/`getSession()` (small, independent pair) → `loginUser()` (most involved, needs both `getUsers()` and `saveSession()` already working) → `logoutUser()` (trivial once `SESSION_KEY` exists).

User

read now , and let me know. Users_key and Session_Key should ne in the .env?

Assistant

No — `.env` is for secrets or things that differ per environment (`VITE_TMDB_KEY`, a future Supabase URL/anon key). These are neither — just internal names for which `localStorage` slot each piece of data lives in, identical in every environment, not sensitive (anyone can see `localStorage` keys in devtools regardless). Keeping them as plain constants in `auth-api.ts` is right.

`auth-api.ts` itself looks correct — `getUsers`, `registerUser`, `loginUser`, `getSession`, `saveSession`, `logoutUser` all do what they're supposed to, including the "email already registered" and "invalid email or password" checks. Two cleanups: `randomAlphaNumeric` now exists in three places (`util.ts` — correct import source, plus a leftover duplicate in `authContext.tsx` lines 21–28, dead code, delete it); bigger one — `authContext.tsx` isn't calling into `auth-api.ts` at all yet, `login`/`logout`/`storedInfo` still read/write `localStorage` directly with key `'user'`, while `auth-api.ts` uses `SESSION_KEY = "session"` — different keys, the two files aren't talking to each other, and since `login` still doesn't call `loginUser()`, it still accepts any credentials.

User

Now I got it. I didnt need Auth-api . is it ?

Assistant

Not strictly "needed" for the app to run — the tutorial proves that, inlining everything directly in the context. Could get a working login/logout without `auth-api.ts` at all. It's there for a reason though: the same split as `tmdb-api.ts` (talks to storage/network) vs `useMovie.ts`/components (React state and UI), and specifically the seam for Supabase later — once wired in, `loginUser`/`registerUser`/`logoutUser` swap their internals from `localStorage` calls to `supabase.auth.*` calls, and `authContext.tsx` doesn't change at all. Without the split, that swap means rewriting logic inside the context itself. Not required, but saves a rewrite in a few weeks.

User

how do I edit auth context them ?

Assistant

Provided pseudocode for the changes: (1) imports — add `loginUser`, `logoutUser`, `getSession` from `auth-api.ts`, drop the local `randomAlphaNumeric`; (2) session restore on load — replace raw `localStorage.getItem('user')` with `getSession()`; (3) `login` — call `loginUser(data)` (which validates, saves the session, and returns an `AuthSession`, and can throw), with two open questions: keep the artificial

`setTimeout` delay or drop it, and should `login` here catch the error or let it bubble up to `signIn.tsx`'s submit handler; (4) `logout` — call `logoutUser()`, reset state, navigate to `/`. Everything else in the file stays as-is.

User

read now

Assistant

One thing got lost in the edit: `AuthContext` itself — the `React.createContext(...)` call — is missing entirely, a hard blocker since `<AuthContext.Provider>` and `useContext(AuthContext)` both reference a name that no longer exists. Fix: add back `export const AuthContext = React.createContext<AuthContextInterface>(initi` right after `initialAuthContext`. Everything else is functionally correct — `login`/`logout` call the right `auth-api.ts` functions, the error from `loginUser()` correctly bubbles up. One small redundancy, not a bug: `loginUser(data)` already returns the `AuthSession`, but `login()` throws that away and calls `getSession()` again for the same data — can just capture what `loginUser()` returns directly.

User

read again

Assistant

Two things broke compared to before — some lines got deleted along with the old TODO comment block. (1) Syntax error: stray extra `}` at line 38, `login`'s arrow function already closes at line 37, delete line 38. (2) Bigger issue: the state/navigate setup got deleted — `login`/`logout` reference `setUser`, `setToken`, `navigate`, and the JSX references `user`/`token`, but none are declared anymore. Needs restoring at the top of `AuthContextProvider`, before `login`: the `storedInfo`/`useState`/`useNavigate` block from two versions ago.

User

read it again

Assistant

Clean — no syntax errors, everything declared, logic is right: session restored from `getSession()` on load, `login` calls `loginUser()` and syncs state + navigates, `logout` calls `logoutUser()` and resets. This closes the loop between `authContext.tsx` and `auth-api.ts`. Only remaining gap: `signIn.tsx`'s form doesn't call `login()` yet — validation logic is there but `onSubmit` isn't wired to `useContext(AuthContext)`.